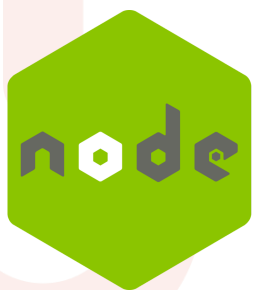




Node.js 基础入门

李喆明





目录

- 01 ▶ 什么是 Node.js
- 02 ▶ Node.js 基础
- 03 ▶ NPM
- 04 ▶ 基于 Node.js 的 Web 开发
- 05 ▶ Node.js 的调试



01

什么是 Node.js



▶ 什么是 Node.js



Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine.

<https://nodejs.org>



与 JavaScript 的区别

- 基于异步 I/O 相关接口
- 基于 node_modules 和 require 的模块依赖
- 提供 C++ addon API 与系统交互



Node.js 可以干什么？

- Web 服务端：Web Server、爬虫
- CLI 命令行脚本：webpack
- GUI 客户端软件：VSCode、网易云音乐
- IoT, 图像处理, 实时通讯, 加密货币...



```
const puppeteer = require('puppeteer');
const url = 'https://movie.douban.com/subject/26794435';

(async () => {
  const browser = await puppeteer.launch();
  const page = await browser.newPage();
  await page.goto(url);
  const film = await page.evaluate(() => {
    const title = $('h1 > span:first-child').text();
    const poster = $('#mainpic img').attr('src');
    const desc = $('span[property="v:summary"]').text().trim();
    return {title, poster, desc};
  });

  console.log(JSON.stringify(film, null, ' '));
  await browser.close();
})();
```

点击重新加载

02 Node.js 基础





Downloads

Latest LTS Version: **14.17.0** (includes npm 6.14.13)

Download the Node.js source code or a pre-built installer for your platform, and start developing today.

LTS Recommended For Most Users	Current Latest Features	
 Windows Installer node-v14.17.0-x86.msi	 macOS Installer node-v14.17.0.pkg	 Source Code node-v14.17.0.tar.gz

Windows Installer (.msi)

Windows Binary (.zip)

macOS Installer (.pkg)

macOS Binary (.tar.gz)

Linux Binaries (x64)

Linux Binaries (ARM)

32-bit	64-bit
32-bit	64-bit
64-bit	
64-bit	
64-bit	
ARMv7	ARMv8

Hello World

```
// index.js  
console.log('Hello World');
```

```
~ node index.js  
Hello World
```

读写文件

```
const fs = require('fs');  
fs.readFile('test.txt', (err, data) => {  
  console.log(data);  
});  
console.log('read file content');
```

模块

- 内置模块：编译进 Node 中，例如 http fs net process path 等
- 文件模块：原生模块之外的模块，和文件(夹)一一对应



使用内置模块

```
const fs = require('fs');  
fs.readFile('a.text', (err, buffer) => {  
  console.log(buffer);  
})
```

```
const {readFile} = require('fs');  
readFile('a.txt', (err, buffer) => {  
  console.log(buffer);  
})
```

使用文件模块

```
// app.js  
var circle = require('./circle.js');  
console.log('半径为4的圆面积是：' + circle.area(4));
```

定义模块

```
// circle.js
const pi = Math.PI;
exports.area = function (r) {
  return pi * r * r;
};
exports.circumference = function (r) {
  return 2 * pi * r;
};
```

模块加载

```
// 加载绝对路径文件  
require('/foo/bar/a.js');
```

```
// 加载相对路径文件  
require('../a.js');
```

```
// 加载无后缀的文件  
require('../a');
```

```
// 加载外部模块  
require('pkg-name');
```

模块类型

- .js
- .json
- .node
- .mjs
- ...



模块路径查找

- 绝对路径
- 相对路径
 - 和当前路径处理为绝对路径
- 模块/文件夹
 - 原生模块，直接读取缓存
 - [`$NODE_PATH`, `~/.node_modules`, `./node_modules`, `../node_modules`, ...]
 - 解析 `package.json`，查找 `main` 属性，没有则使用 `index.js`
 - 如果未找到，则报错



js 模块解析

js 模块解析

```
// app.js  
const circle = require('./circle.js');
```

js 模块解析

```
// app.js  
const circle = require('./circle.js');
```

- `require` 并不是全局变量
- 定义的变量 `circle` 会污染其他文件么？

js 模块解析

- 通过 fs.readFileSync 同步拿到文件内容
- 对内容进行包装

```
(function (exports, require, module, __filename, __dirname) {  
    var circle = require('./circle.js');  
    console.log('The area is ' + circle.area(4));  
});
```

- 通过 vm.runInThisContext 执行
- 获取 module 对象的值作为模块的返回值

模块缓存

- 模块加载后会将返回值缓存起来
- 下次加载时直接读取缓存结果，避免文件 I/O 和解析时间
- 导出对象缓存在 `Module._cache` 对象上



02 Node Package Manager

<https://code.h5jun.com/dac>



► NPM 包管理器



各包管理模块数对比

截止至 2020年03月24日

***数据来源: **

- <https://npmjs.com>
- <http://pypi.douban.com/simple>
- <https://packagist.org/statistics>
- <http://bestgems.org/stat/gems>



包管理

- 一个package.json文件应该存在于包顶级目录下
- 二进制文件应该包含在bin目录下
- JavaScript代码应该包含在lib目录下
- 文档应该在doc目录下
- 单元测试应该在test目录下



package.json

```
star-plan npm init -y
```

Wrote to /Users/lizheming/star-plan/package.json:

```
{  
  "name": "star-plan",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

Tip: 可以使用 `npm config set init.author.name` 等命令
修改初始化时的默认值

main

npm scripts

1. dependencies

2. devDependencies

3. peerDependencies

4. bundledDependencies

5. optionalDependencies

semver version

bin

registry

包依赖

```
"dependencies": {  
  "accepts": "^1.2.2",  
  "content-disposition": "~0.5.0",  
  "cookies": "~0.7.0",  
  "debug": "*",  
  "delegates": "^1.0.0",  
  "escape-html": "~1.0.1",  
  "fresh": "^0.5.2",  
  "only": "0.0.2",  
  "parseurl": "^1.3.0",  
  "statuses": "^1.2.0",  
  "type-is": "^1.5.5",  
  "vary": "^1.0.0"  
},
```

包依赖

- **1.0.0** Must match version exactly
- **>1.0.0** Must be greater than version
- **>=1.0.0 <1.0.0 <=1.0.0**
- **~1.0.0** "Approximately equivalent to version"
- **^1.0.0** "Compatible with version"
- **1.2.x** 1.2.0, 1.2.1, etc., but not 1.3.0
- ***** Matches any version
- **version1 – version2** Same as **>=version1 <=version2**.
- ...





NPM 的问题?

1. 速度问题
2. 安全问题

- 查看源码看 Star
- <https://snyk.io/> 外部的检测服务
- npm audit



03 基于 Node.js 的Web 开发



Web

```
const http = require('http');  
const server = http.createServer((req, res) => {  
  res.end('Hello World');  
});  
server.listen(3000);
```

Koa

<https://github.com/koajs/koa>

```
const Koa = require('koa');
const app = new Koa();

// response
app.use(ctx => {
  ctx.body = 'Hello Koa';
});

app.listen(3000);
```

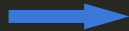
Koa

<https://github.com/koajs/koa>

```
const Koa = require('koa');  
const app = new Koa();
```

```
// response  
app.use(ctx => {  
  ctx.body = 'Hello Koa';  
});
```

```
app.listen(3000);
```



```
module.exports = class Application extends Emitter {  
  ...
```

```
  listen() {  
    debug('listen');  
    const server = http.createServer(this.callback());  
    return server.listen.apply(server, arguments);  
  }
```

```
  use(fn) {  
    this.middleware.push(fn);  
    return this;  
  }
```

```
  callback() {  
    const fn = compose(this.middleware);
```

```
    if (!this.listeners('error').length) this.on('error', this.onerror);
```

```
    const handleRequest = (req, res) => {  
      res.statusCode = 404;  
      const ctx = this.createContext(req, res);  
      const onerror = err => ctx.onerror(err);  
      const handleResponse = () => respond(ctx);  
      onFinished(res, onerror);  
      return fn(ctx).then(handleResponse).catch(onerror);  
    };
```

▶ Node.js Web 开发



- 逻辑分层
- 路由处理
- 数据解析、校验
- 权限校验
- Session、Cache
- 数据库、Redis
- 安全
- ...

Koa 无规范约束，不利于团队开发
中间件繁多，质量参差不齐，选择困难



```
├──src
│   ├──bootstrap
│   ├──config
│   │   ├──config.js
│   │   └──adapter.js
│   ├──controller
│   │   ├──index.js
│   └──logic
│       ├──index.js
│       └──model
├──view
│   └──index_index.html
├──www
│   ├──static
│   │   ├──css
│   │   ├──img
│   │   └──js
├──development.js
├──production.js
└──package.json
```



点击重新加载

<https://thinkjs.org>

```
// src/controller/index.js
export default class extends think.Controller {
  indexAction() {
    return this.display();
  }
}

// view/index_index.html
<h3>Hello World</h3>
```



TODO List 项目实战

<https://github.com/lizheming/simple-todo>



The screenshot shows a web application for managing a TODO list. At the top, there is a green input field with the placeholder text "Enter an activity.." and a white circular button with a plus sign. Below this, the list contains three items, each in a white box with a light gray border. The first item is "周六下午跑步" (Run on Saturday afternoon), the second is "十点去菜市场买菜记得多买点肉" (Go to the vegetable market at 10 o'clock, remember to buy more meat), and the third is "八点打扫房间" (Clean the room at 8 o'clock). Each item has a trash icon and a green checkmark icon to its right. A horizontal line separates the second and third items.



功能列表

- TODO List 的页面
- API
 - 获取 TODO 列表
 - 增加 TODO
 - 删除 TODO
 - 更新 TODO 状态



数据表设计

```
CREATE TABLE `todo` (  
  `id` int(11) unsigned NOT NULL AUTO_INCREMENT,  
  `desc` varchar(255) NOT NULL DEFAULT "",  
  `status` tinyint(11) NOT NULL DEFAULT '0' COMMENT '0 是未完成，1是已完成',  
  `createdAt` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updatedAt` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
npm install -g think-cli
```

```
~ thinkjs --version
```

```
2.2.8
```

创建项目

```
$ thinkjs new todo
```

```
$ cd todo
```

```
$ npm install
```

- ▶ node_modules
- ▶ runtime
- ▶ src
 - ▶ bootstrap
 - ▶ config
- ▶ controller
 - base.js
 - index.js
- ▶ logic
- ▶ model
- ▶ test
- ▶ view
- .eslintrc
- .gitignore
- development.js
- nginx.conf
- package-lock.json

启动项目

simple-todo npm **start**

```
> simple-todo@1.0.0 start /Users/lizheming/Desktop/star-plan/simple-todo  
> node development.js
```

```
[2019-04-21T21:58:16.197] [7077] [INFO] – Server running at http://127.0.0.1:8361  
[2019-04-21T21:58:16.201] [7077] [INFO] – ThinkJS version: 3.2.10  
[2019-04-21T21:58:16.201] [7077] [INFO] – Environment: development  
[2019-04-21T21:58:16.201] [7077] [INFO] – Workers: 1
```

A New App Created By ThinkJS

1

Generate Files

Run `thinkjs` command to create module, controler, model, service and so on.

2

Documentation

ThinkJS has online html documents. visit <https://thinkjs.org/doc.html>.

3

GitHub

If you have some questions, please [new a issue](#).

模板渲染

```
// src/controller/index.js
const Base = require('./base.js');
module.exports = class extends Base {
  indexAction() {
    return this.display();
  }
};
```

API 开发

使用 RESTful API



RESTful 接口规范

1. 每个 API 都对应一种资源或资源集合
2. 使用 HTTP Method 来表示对资源的动作
3. 使用 HTTP Status Code 来表示资源操作结果



RESTful API

- GET /ticket 获取 ticket 列表
- GET /ticket/:id 查看某个具体的 ticket
- POST /ticket 新建一个 ticket
- PUT /ticket/:id 更新 id 为 12 的 ticket
- DELETE /ticket/:id 删除 id 为 12 的 ticket



创建 API 文件

```
simple-todo thinkjs controller -r ticket
```

```
think-cli · Create: src/controller/rest.js
```

```
think-cli · Create: src/controller/ticket.js
```

```
think-cli · Create: src/logic/api/ticket.js
```

配置路由

```
// src/config/router.js  
module.exports = [  
  ['/ticket/:id?', 'rest'], // 配置 RESTful API 路由  
]
```

路由解析

- **GET** /api/todo 获取 TODO 列表，执行 `getAction`
- **GET** /api/todo/:id 获取某个TODO的详细信息，执行 `getAction`
- **POST** /api/todo 添加一个 TODO，执行 `postAction`
- **PUT** /api/todo/:id 更新一个 TODO，执行 `putAction`
- **DELETE** /api/todo/:id 删除一个 TODO，执行 `deleteAction`



getAction

```
// src/controller/rest.js
async getAction() {
  let data;
  if (this.id) {
    const pk = this.modelInstance.pk;
    data = await this.modelInstance.where({ [pk]: this.id }).find();
    return this.success(data);
  }
  data = await this.modelInstance.select();
  return this.success(data);
}
```

postAction

```
async postAction() {  
  const pk = this.modelInstance.pk;  
  const data = this.post();  
  delete data[pk];  
  if (think.isEmpty(data)) {  
    return this.fail('data is empty');  
  }  
  const insertId = await this.modelInstance.add(data);  
  return this.success({ id: insertId });  
}
```

deleteAction

```
async deleteAction() {  
  if (!this.id) {  
    return this.fail('params error');  
  }  
  const pk = this.modelInstance.pk;  
  const rows = await this.modelInstance.where({ [pk]: this.id }).delete();  
  return this.success({ affectedRows: rows });  
}
```

putAction

```
async putAction() {  
  if (!this.id) {  
    return this.fail('params error');  
  }  
  const pk = this.modelInstance.pk;  
  const data = this.post();  
  delete data[pk];  
  if (think.isEmpty(data)) {  
    return this.fail('data is empty');  
  }  
  const rows = await this.modelInstance.where({ [pk]: this.id }).update(data);  
  return this.success({ affectedRows: rows });  
}
```

数据库配置

```
// src/config/adapter.js
exports.model = {
  type: 'mysql',
  common: {
    logConnect: isDev,
    logSql: isDev,
    logger: msg => think.logger.info(msg)
  },
  mysql: {
    handle: mysql,
    database: 'todo',
    prefix: '',
    encoding: 'utf8',
    host: '127.0.0.1',
    port: '',
    user: 'root',
```

http://127.0.0.1:8360/ticket

```
{
  "errno": 0,
  "errmsg": "",
  "data": [{
    "id": 1,
    "desc": "八点打扫房间",
    "status": 0,
    "createdAt": "2018-07-08 17:12:59",
    "updatedAt": "2018-07-08 17:13:14"
  }]
}
```

数据校验

- 提供了 Logic 机制专门用来支持数据校验
- 文件和 Action 与 Controller 一一对应



数据校验

```
// src/logic/ticket.js
module.exports = class extends think.Logic {
  getAction() {
    this.rules = {
      id: {
        int: true
      }
    };
  }
  deleteAction() {
    this.rules = {
      id: {
        int: true,
        required: true,
        method: 'get'
      }
    };
  }
}
```

DELETE <http://127.0.0.1:8360/ticket>

```
{  
  "errno": 1001,  
  "errmsg": {  
    "id": "id can not be blank"  
  }  
}
```

数据库操作

- 封装了 think.Model 类
- 提供增删改查等操作
- 支持关联模型查询
- 自动分析数据表字段类型
- 自动数据安全过滤



控制器中操作模型

```
const model = this.model(modeName);
```

- 根据模型名查找 src/model 下的模型文件
 - 文件存在，实例化对应的模型类
 - 文件不存在，实例化 think.Model 类

定义模型类

```
// src/model/todo.js
```

```
module.exports = class TodoModel extends think.Model {  
  getList () {  
    // get list  
  }  
}
```

模型的好处

- 简化代码、提高效率
- 不用太懂 SQL 语句也能操作数据库
- 避免手写 SQL 语句的安全风险



05 Node.js 的调试



NodeJS调试

<https://zhuanlan.zhihu.com/p/41315709>

- 日志调试
- 断点调试
 - node --inspect
 - vscode
 - ndb



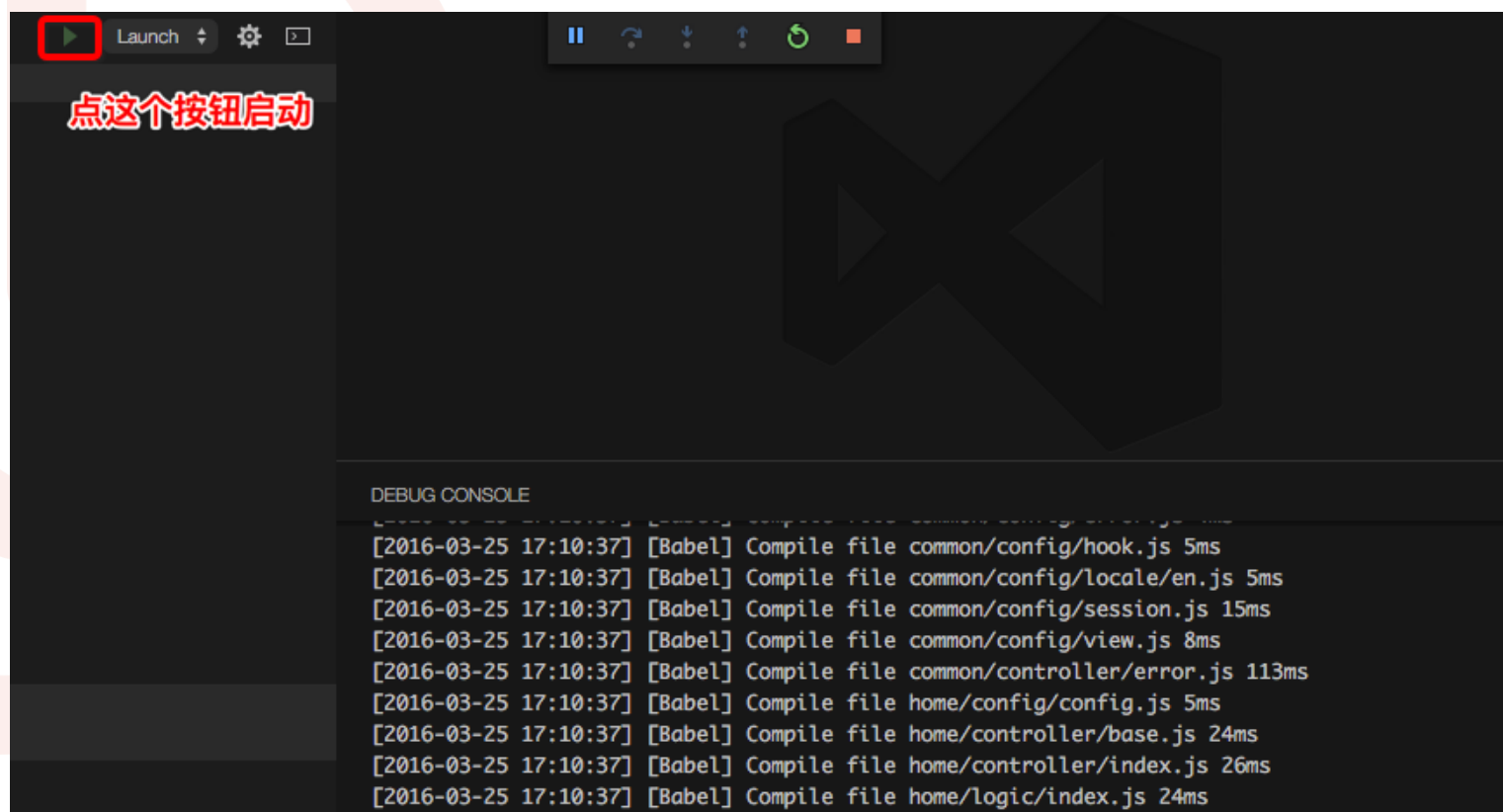
Node.js 调试



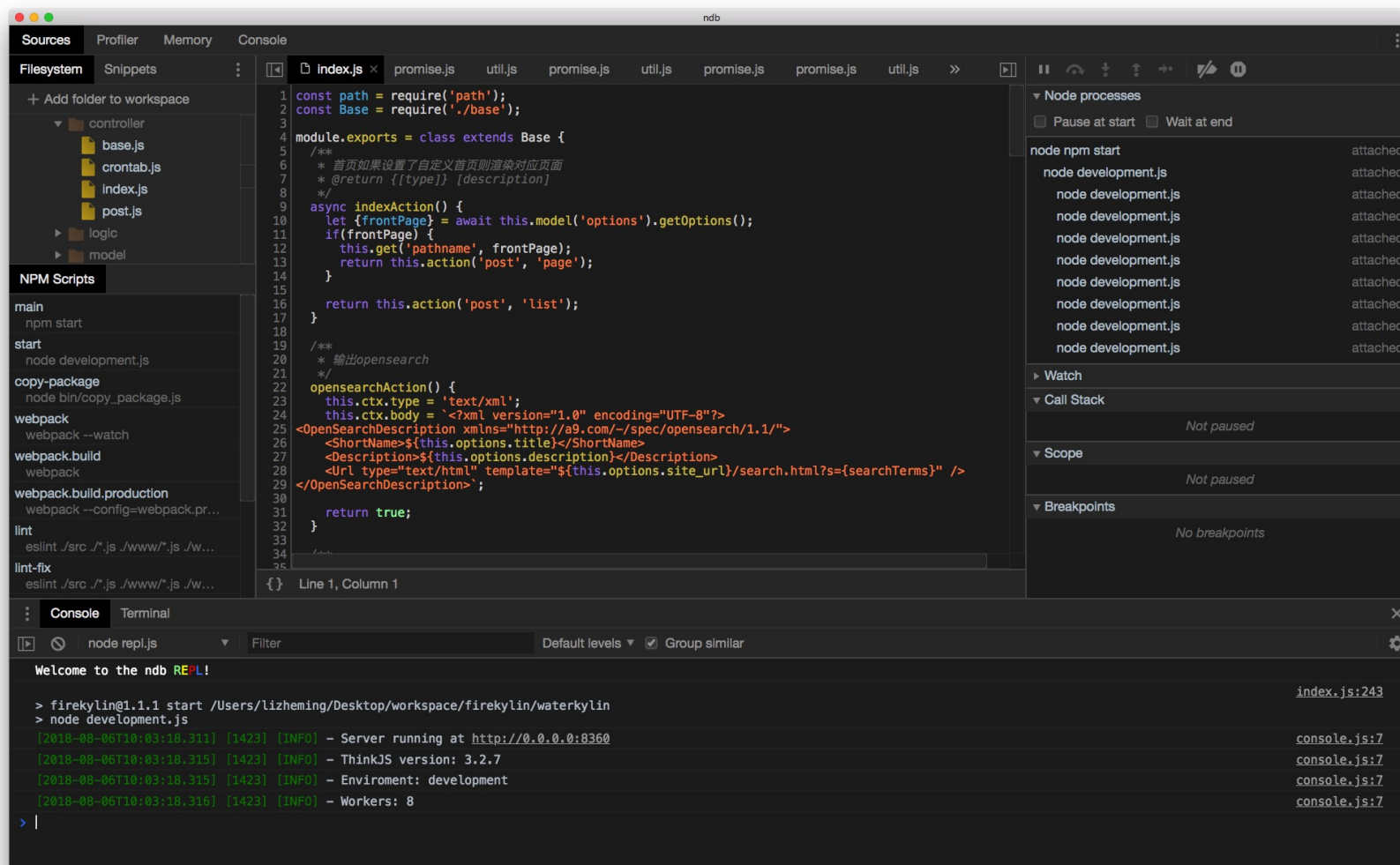
NodeJS 6.3+ 使用 `node --inspect` 参数启动可以在 Chrome 浏览器中调试，在 `chrome://inspect` 中可以发现你的项目并启动 devtool



Node.js 调试



Node.js 调试



Node 开发角色转换

- 前端

- 跟浏览器打交道，兼容性问题
- 组件化
- 加载速度、JS 执行性能、渲染性能
- 错误监控
- XSS、CSRF 等安全漏洞

- 服务端

- 数据库、Redis 等周边服务
- 性能、内存泄露、CPU、机器管理
- 服务监控、错误监控、流量监控、报警
- SQL注入、目录遍历等安全漏洞



谢谢聆听



前端星计划

